전북지역혁신플랫폼
Jeonbuk Regional Innovation Platform

JST

지능형모빌리티 설계

5주차 ROS2 프로그래밍 #3

1차시 Launch 및 파라미터



* 본 영상은 무단 배포할 수 없습니다.

학습내용

1 Launch 파일 생성 방법

2 파라미터 관리 방법

학습목표

🎯 Launch 파일을 사용할 수 있다.

🎯 파라미터를 효율적으로 관리할 수 있다.

Launch 파일 생성 방법

Launch

Launch 개요

- 여러 Node를 한 번에 실행시키기 위한 명령어
- XML, python 형태로 작성 가능
- Launch 디렉토리 및 파일 생성

```
$ cd ~/ros2_ws/src/ros2_cmake/
$ mkdir launch && cd launch
$ code topic_launch.py
```

Linux

Launch 파일 작성

- Topic 예제 Launch로 실행
- Topic 챕터에서 생성한 노드를 불러와서 실행하도록 함

Linux

topic_launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node
def generate_launch_description():
    return LaunchDescription([
        Node(
            package='ros2_cmake',
            namespace='ros2_cmake',
            executable='publisher',
            name='ex_pub'
        ),
        Node(
            package='ros2_cmake',
            namespace='ros2_cmake',
            executable='subscriber',
            name='ex_sub'
        ),
    ])
```

패키지 이름

실행 파일

Node 이름

실행 파일

Launch 파일

CMakeList.txt

```
# launch file
install(DIRECTORY 빌드 후 해당 디렉토리를 지정된 위치로 복사
 launch
 DESTINATION share/${PROJECT_NAME} 지정된 위치
)
```

Linux

Build

```
$ cd ~/ros2_ws/
$ colcon build --packages-select ros2_cmake
$ source install/setup.bash
```

Linux

실행

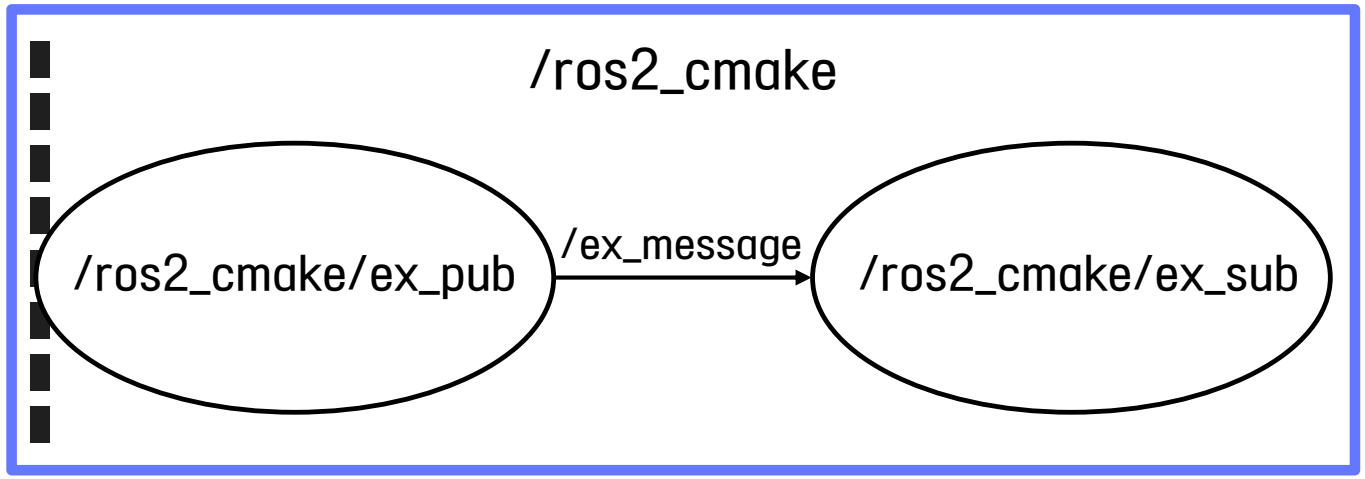
```
$ ros2 launch ros2_cmake topic_launch.py
```

출력

```
[publisher-1] [INFO] [1735280758.738407187]
[ros2_cmake.ex_pub]: Publishing: 'Hello, world! 0'
[subscriber-2] [INFO] [1735280758.739067397]
[ros2_cmake.ex_sub]: I heard: 'Hello, world! 0'
[publisher-1] [INFO] [1735280759.238428151]
[ros2_cmake.ex_pub]: Publishing: 'Hello, world! 1'
```

Linux

rqt_graph



파라미터 관리 방법

파라미터 - cpp

Service에 파라미터 추가

- Service_client.cpp 파일에 파라미터를 추가
- param_client.cpp 파일을 새로 만들어서 작성
- param_client.cpp

Service_client.cpp와 동일

```
#include "rclcpp/rclcpp.hpp"
#include "ros2_interface/srv/ex_custom_srv.hpp"
#include <chrono>
#include <cstdlib>
#include <memory>

using namespace std::chrono_literals;

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
```

파라미터 - cpp

param_client.cpp

프로그램 실행 시 인자로 받는 부분 삭제

```
std::shared_ptr<rclcpp::Node> node = rclcpp::Node::make_shared("param_client");
rclcpp::Client<ros2_interface::srv::ExCustomSrv>::SharedPtr client =
    node->create_client<ros2_interface::srv::ExCustomSrv>("add_two_ints");

const int invalid_value = std::numeric_limits<int>::min(); // 특수한 기본값(유효하지 않은 값으로 고려)
int num1 = invalid_value;
int num2 = invalid_value;
std::string param_name1 = "number1";
std::string param_name2 = "number2";

// 파라미터 선언
node->declare_parameter(param_name1,rclcpp::PARAMETER_INTEGER);
node->declare_parameter(param_name2, rclcpp::PARAMETER_INTEGER);
```

파라미터 - cpp

param_client.cpp

```
// 파라미터 값 얻기 (number1)
    if (!node->get_parameter(param_name1, num1)) {
        RCLCPP_WARN(node->get_logger(), "Parameter '%s' not found, using default invalid value",
param_name1.c_str());
    }

    // 파라미터 값 얻기 (number2)
    if (!node->get_parameter(param_name2, num2)) {
        RCLCPP_WARN(node->get_logger(), "Parameter '%s' not found, using default invalid value",
param_name2.c_str());
    }
```

파라미터 - cpp

param_client.cpp

```
// 유효성 검사
    // 두 숫자 파라미터를 모두 얻었는지 확인
    if (num1 == invalid_value || num2 == invalid_value) {
        RCLCPP_ERROR(node->get_logger(), "Invalid parameters detected. num1 = %d, num2 = %d. Check
your parameter settings.", num1, num2);
        rclcpp::shutdown();
        return 1;
    }

    auto request = std::make_shared<ros2_interface::srv::ExCustomSrv::Request>();
    request->a = num1;
    request->b = num2;
```

파라미터 - cpp

param_client.cpp

```
while (!client->wait_for_service(1s)) {
      if (!rclcpp::ok()) {
      RCLCPP_ERROR(rclcpp::get_logger("rclcpp"), "Interrupted while waiting for the service. Exiting.");
      return 0;
      }
      RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "service not available, waiting again...");
   }

   auto result = client->async_send_request(request);
   // Wait for the result.
   if (rclcpp::spin_until_future_complete(node, result) ==
       rclcpp::FutureReturnCode::SUCCESS)
   {
       RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "Sum: %ld", result.get()->sum);
   } else {
       RCLCPP_ERROR(rclcpp::get_logger("rclcpp"), "Failed to call service add_two_ints");
   }

   rclcpp::spin(node);
   rclcpp::shutdown();
   return 0;
}
```

Client가 Response를 받은 후에도 종료되지 않도록

파라미터 - cpp

Launch 파일에 파라미터 추가

param_launch.py 파일 생성

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='ros2_cmake',
            namespace='ros2_cmake',
            executable='service_server',
            name='server'
        ),
```

ros2_cmake 패키지

service_server 실행 파일

파라미터 - cpp

Launch 파일에 파라미터 추가

param_launch.py 파일 생성

```
Node(
            package='ros2_cmake',
            namespace='ros2_cmake',
            executable='param_client',
            name='client',
            parameters=[
                {"number1" : 1 },
                {"number2" : 2 }
            ]
        ),
    ])
```

param_client 실행 파일

파라미터 설정

파라미터 - cpp

CMakeList.txt 파일 변경

```
# parameter
add_executable(param_client src/param_client.cpp)
ament_target_dependencies(param_client rclcpp ros2_interface)

install(TARGETS
 ...
 #parameter
 param_client

 DESTINATION lib/${PROJECT_NAME})


```

파라미터 - cpp

Build

```
$ cd ~/ros2_ws/
$ colcon build --packages-select ros2_cmake --symlink-install
$ source install/setup.bash
```

- Launch 파일 수정 시 다시 빌드하지 않아도 됨

파라미터 - cpp

실행

터미널

```
$ ros2 launch ros2_cmake param_launch.py
```

터미널 출력

```
[service_server-1] [INFO] [1736338364.127533394] [rclcpp]: Ready to add two ints.
[service_server-1] [INFO] [1736338364.127669164] [rclcpp]: Incoming request
[service_server-1] a: 1 b: 2
[service_server-1] [INFO] [1736338364.127677684] [rclcpp]: sending back response: [3]
[param_client-2] [INFO] [1736338364.127783842] [rclcpp]: Sum: 3
[INFO] [param_client-2]: process has finished cleanly [pid 6993]
```

- Launch 파일에서 파라미터를 다른 값으로 수정하면 다른 결과를 얻을 수 있음

파라미터 - cpp

파라미터 파일

- 파라미터가 많아지면 Launch 파일에 파라미터를 작성하면 관리가 어려움
- 파라미터에 관한 파일을 생성해 해당 파일에서 관리
- 파라미터 디렉토리 및 파일 생성

```
$ cd ~/ros2_ws/src/ros2_cmake/
$ mkdir config && cd config
$ code param.yaml
```

파라미터 - cpp

파라미터 파일

param.yaml

```
ros2_cmake: Namespace
 client: Node 이름
   ros__parameters: ROS 파라미터임을 나타냄
     number1: 5
     number2: 15
```

파라미터 이름: 설정값

파라미터 - cpp

Launch 파일에 yaml 추가

param_launch.py 파일 변경

```
from launch import LaunchDescription
from launch_ros.actions import Node

# yaml 파일에서 파라미터 읽어오기
from ament_index_python.packages import get_package_share_directory
from launch.substitutions import PathJoinSubstitution

config_file = PathJoinSubstitution([get_package_share_directory('ros2_cmake'), 'config', 'param.yaml'])
```

yaml 파일을 읽어 오기 위해

파라미터 설정된 파일

패키지 이름 디렉토리 파라미터 파일 이름

파라미터 - cpp

Launch 파일에 yaml 추가

param_launch.py 파일 생성

```
def generate_launch_description():
    return LaunchDescription([
        Node(
            package='ros2_cmake',
            namespace='ros2_cmake',
            executable='service_server',
            name='server'
        ),
        Node(
            package='ros2_cmake',
            namespace='ros2_cmake',
            executable='param_client',
            name='client',
            parameters=[
                config_file
            ]
        ),
    ])
```

namespace 이름을 파라미터 파일과 동일하게 작성

Node 이름을 파라미터 파일과 동일하게 작성

위에서 찾은 파라미터 파일

파라미터 - cpp

CMakeList.txt 파일 변경

```
# launch file
install(DIRECTORY
 launch
 config
 DESTINATION share/${PROJECT_NAME}
)
```

config 디렉토리도 지정된 위치하도록

파라미터 - cpp

Build

```
$ cd ~/ros2_ws/
$ colcon build --packages-select ros2_cmake --symlink-install
$ source install/setup.bash
```

파라미터 - cpp

실행

터미널

```
$ ros2 launch ros2_cmake param_launch.py
```

터미널 출력

```
[service_server-1] [INFO] [1736339682.587946510] [rclcpp]: Ready
to add two ints.
[service_server-1] [INFO] [1736339682.589081695] [rclcpp]:
Incoming request
[service_server-1] a: 5 b: 15
[service_server-1] [INFO] [1736339682.589092661] [rclcpp]: sending
back response: [20]
[param_client-2] [INFO] [1736339682.589191980] [rclcpp]: Sum: 20
```

- 파라미터 파일에서 파라미터를 다른 값으로 수정하면 다른 결과를 얻을 수 있음

파라미터 - python

Launch 및 파라미터 디렉토리, 파일 생성

```
$ cd ~/ros2_ws/src/ros2_python/
$ mkdir launch && cd launch
$ code param_launch.py
```

```
$ cd ~/ros2_ws/src/ros2_python/
$ mkdir param && cd param
$ code param.yaml
```

파라미터 - python

param_client.py

param_client.py를 새롭게 생성

```
import sys
from ros2_interface.srv import ExCustomSrv
import rclpy
from rclpy.node import Node

class CustomClientAsync(Node):

    def __init__(self):
        super().__init__('minimal_client_async')
        self.cli = self.create_client(ExCustomSrv, 'add_two_ints')
        while not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('service not available, waiting again...')
        self.req = ExCustomSrv.Request()

        # 파라미터 선언
        self.declare_parameter('number1', 0)
        self.declare_parameter('number2', 0)
```

파라미터 - python

param_client.py

param_client.py를 새롭게 생성

```
def send_request(self, a, b):
        self.req.a = a
        self.req.b = b
        return self.cli.call_async(self.req)


def main():
    rclpy.init()

    custom_client = CustomClientAsync()

    # 파라미터 값 읽기
    num1 = custom_client.get_parameter('number1').value
    num2 = custom_client.get_parameter('number2').value
```

파라미터 얻기

파라미터 - python

param_client.py

param_client.py를 새롭게 생성

```
future = custom_client.send_request(int(num1), int(num2))
    rclpy.spin_until_future_complete(custom_client, future)

    response = future.result()
    custom_client.get_logger().info(
        'Result of add_two_ints: for %d + %d = %d' %
        (int(num1), int(num2), response.sum))

    custom_client.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

파라미터 - python

Launch 파일에 yaml 추가

param_launch.py 파일 변경

```
from launch import LaunchDescription
from launch_ros.actions import Node

# yaml 파일에서 파라미터 읽어오기
from ament_index_python.packages import get_package_share_directory
from launch.substitutions import PathJoinSubstitution

config_file = PathJoinSubstitution([get_package_share_directory('ros2_python'), 'param', 'param.yaml'])
```

yaml 파일을 읽어 오기 위해

파라미터 설정된 파일 — 패키지 이름 — 디렉토리 — 파라미터 파일 이름

파라미터 - python

Launch 파일에 yaml 추가

param_launch.py 파일 생성

```
def generate_launch_description():
    return LaunchDescription([
        Node(
            package='ros2_python',
            namespace='ros2_python',
            executable='service_server',
            name='server'
        ),
        Node(
            package='ros2_python',
            namespace='ros2_python',
            executable='param_client',
            name='client',
            parameters=[
                config_file
            ]
        ),
    ])
```

namespace 이름을 파라미터 파일과 동일하게 작성

Node 이름을 파라미터 파일과 동일하게 작성

위에서 찾은 파라미터 파일

파라미터 - python

파라미터 파일

param.yaml

```
ros2_python: namespace
  client: Node 이름
    ros__parameters: ROS 파라미터임을 나타냄
      number1: 5
      number2: 15
```

파라미터 이름: 설정값

파라미터 - python

setup.py

```
from setuptools import find_packages, setup
from glob import glob
import os
package_name = 'ros2_python'
setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        ('share/' + package_name + '/launch', glob(os.path.join('launch', '*.py'))),
        ('share/' + package_name + '/param', glob(os.path.join('param', '*.yaml'))),
    ],
```

Launch, 파라미터 파일을 찾기 위해

Launch와 파라미터 파일 경로

파라미터 - python

setup.py

```
entry_points={
        'console_scripts': [
                        ...
            'param_client = ros2_python.param_client:main',
        ],
    },
```

파라미터 - python

Build

```
$ cd ~/ros2_ws/
$ colcon build --packages-select ros2_python --symlink-install
$ source install/setup.bash
```

파라미터 - python

실행

터미널

```
$ ros2 launch ros2_python param_launch.py
```

터미널 출력

```
[service_server-1] [INFO] [1736342514.332231513]
[ros2_python.server]: Incoming request
[service_server-1] a: 5 b: 15
[param_client-2] [INFO] [1736342514.340702076]
[ros2_python.client]: Result of add_two_ints: for 5 + 15 = 20
[INFO] [param_client-2]: process has finished cleanly [pid 11363]
```